Checkpoint 2: TCP Reno

15-441/641 - TCP in the Wild

This report evaluates the implemented Reno sender using transfers between two separate AWS EC2 instances. All figures are generated from recorded runs; the accompanying CSV files retain timings, checksums, network settings, and congestion events.

Implementation

The sender limits outstanding bytes to min(cwnd, rwnd). Slow start increases cwnd by up to one MSS per new data ACK; congestion avoidance adds MSS squared divided by cwnd. Three duplicate ACKs halve the threshold, inflate cwnd by three MSS, and retransmit the oldest missing segment. Further duplicate ACKs inflate cwnd by one MSS. The next advancing ACK exits fast recovery at the threshold. A timeout halves the threshold and resets cwnd to one MSS. This is Reno, without NewReno partial-ACK recovery or selective acknowledgments.

Send buffering includes queued and unacknowledged bytes. Ordered and reordered receive data share MAX_NETWORK_BUFFER. The receive window describes a bounded sequence interval, allowing reordered bytes to occupy existing positions without shrinking the right edge. Zero-window probes and read-triggered window reopening prevent permanent flow-control stalls. SYN/FIN reliability and independent close in each direction remain supported.

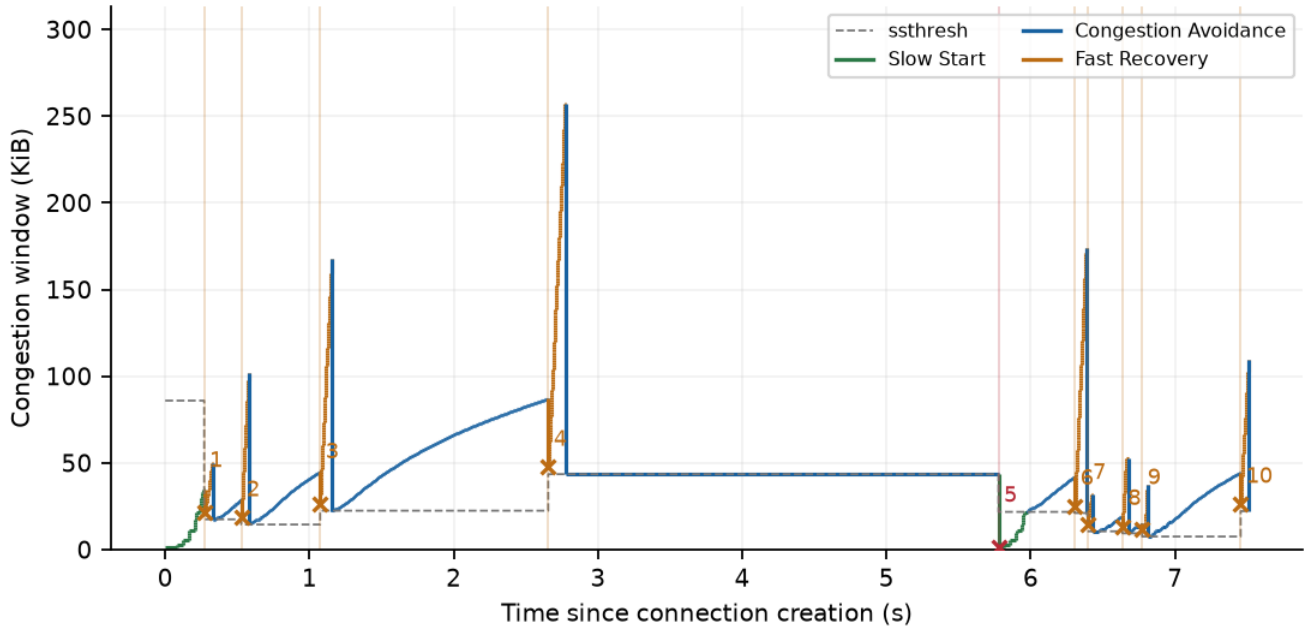
Experimental configuration

Setting	Value
Placement	Two Ubuntu 24.04 t3.micro instances, us-west-2a
Path	Private IPv4, same VPC and availability zone
Rate / delay	5 Mbps and 20 ms egress delay on each node (~40 ms RTT)
Sender egress loss	0.1%, 0.2%, 0.5%, 1%, 2%, 5%, 10%; receiver loss 0%
Repetitions / file	10 at each rate; 2,097,152 bytes per transfer
Protocol parameters	MSS 1375 B; initial cwnd 1375 B; initial ssthresh 88,000 B
Buffer / timeout	65,535 bytes per direction; fixed RTO 3000 ms

Timing starts just after socket creation and ends when an application receipt confirms that the receiver has written all file bytes. It includes the remaining connection establishment and final receipt; FIN and TIME_WAIT are excluded. SHA-256 is compared after every transfer. The random payload and file size are identical across all trials.

Congestion-window behavior

Selected connection: 0.2% sender loss, trial 8, 2,097,152 bytes. Transfer time: 7.512 s. The full trace contains 9 triple-duplicate-ACK events and 1 timeout. Colors identify the sender state after each event; numbered marks identify every loss response.



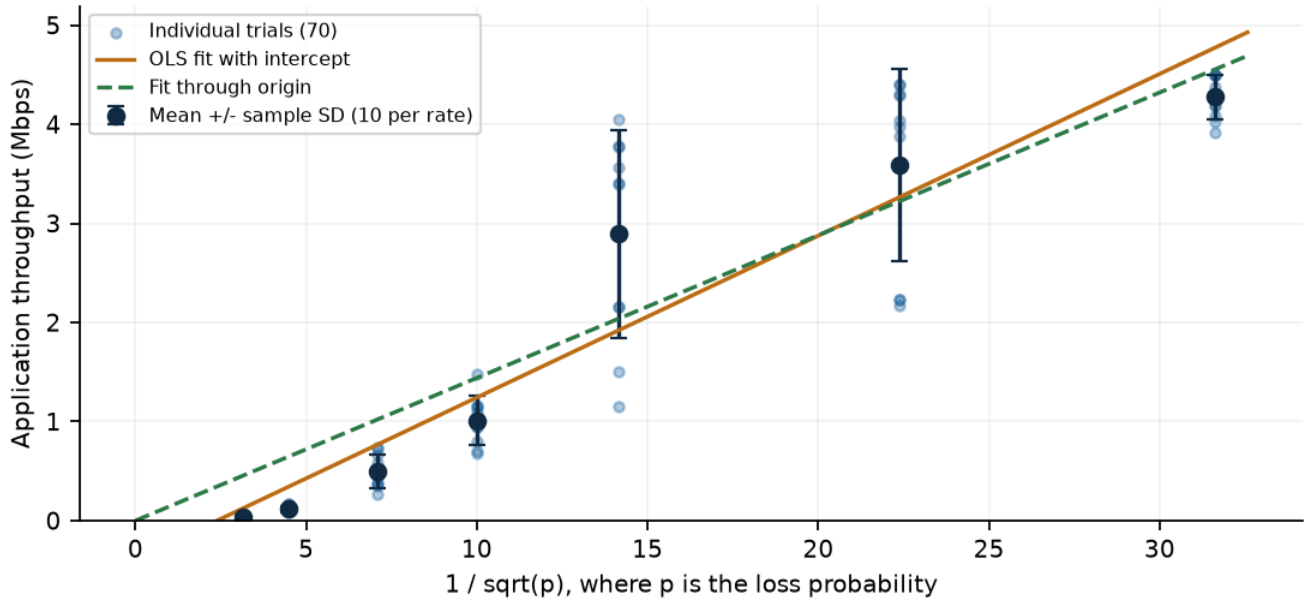
Mark	Time (s)	Event	cwnd after (B)	ssthresh (B)
1	0.270	3 duplicate ACKs	22,000.0	17,875.0
2	0.527	3 duplicate ACKs	18,926.3	14,801.3
3	1.074	3 duplicate ACKs	27,102.4	22,977.4
4	2.655	3 duplicate ACKs	48,683.3	44,558.3
5	5.776	Timeout	1,375.0	22,279.2
6	6.306	3 duplicate ACKs	25,730.6	21,605.6
7	6.392	3 duplicate ACKs	14,927.8	10,802.8
8	6.635	3 duplicate ACKs	13,757.5	9,632.5
9	6.772	3 duplicate ACKs	11,828.4	7,703.4
10	7.455	3 duplicate ACKs	26,835.1	22,710.1

Slow start grows rapidly; congestion avoidance produces the additive-growth ramp. The tall fast-recovery peaks reflect duplicate-ACK window inflation, while actual outstanding data remains bounded by rwnd. The next advancing ACK deflates cwnd to ssthresh. A timeout restarts from one MSS when ACK feedback is insufficient for fast recovery. Multiple losses in a Reno window can therefore produce additional timeouts.

Source trace: loss-0.2-run-08.csv

Throughput and the Mathis relationship

Let p be the configured independent loss probability, expressed as a fraction. For fixed MSS and RTT, the Mathis approximation predicts throughput proportional to x = 1/sqrt(p). The plot shows all 70 measured application throughputs; error bars are the sample standard deviation within each group of ten.



Loss (%)	Mean (Mbps)	Sample SD	Mean duration (s)	Timeouts (total)
0.1	4.27626	0.22526	3.93	0
0.2	3.59226	0.97069	5.09	3
0.5	2.89308	1.04914	6.89	7
1	1.01111	0.24706	17.53	38
2	0.49839	0.16848	37.44	97
5	0.11646	0.02989	151.48	458
10	0.03853	0.00360	438.71	1394

Ordinary least squares over all trials gives T = 0.163281 x -0.390209 Mbps, with Pearson r = 0.91302 and R squared = 0.83361. The correlation of the seven group means is 0.95959. A fit constrained through the origin gives T = C/sqrt(p), with C = 0.144013 Mbps. Under the dimensionless convention T = C*MSS/(RTT*sqrt(p)), using MSS = 1375 bytes and RTT = 40 ms, C = 0.52368.

Interpretation and validation

The group means show strong positive agreement with the inverse-square-root trend (r = 0.95959). This supports the trend to that extent, but it does not establish the Mathis approximation as an exact model of these transfers. The reported intercept and trial variability quantify departures from a pure proportional relationship.

For an exact proportional law, T*sqrt(p) would be constant. Here the seven normalized means range from 0.01219 to 0.20457 Mbps, a factor of 16.79. The pure proportional model does not adequately describe the entire loss range. A high correlation alone is insufficient to establish this stronger claim.

At low loss, the 5 Mbps rate limit, queueing delay, receive window, and finite slow-start period can cap or reduce throughput. At higher loss, multiple missing packets and the fixed three-second retransmission timeout make Reno spend time waiting rather than following the steady-state sawtooth assumed by the approximation. These effects can lower throughput relative to a model based only on p and a nominal 40 ms RTT. The configured RTT is a baseline; serialization and queueing increase actual RTT during a transfer.

For the ten 10% loss trials, the number of recorded timeouts multiplied by the fixed 3 s RTO accounts for approximately 95.3% of total transfer time. This estimates the contribution of timeout intervals; it is not a direct measurement of link idleness.

Transfer durations range from 3.728 to 487.274 seconds. All 70 files passed the SHA-256 check. No trials failed. No missing data is interpolated. The experiment uses application goodput, so transport headers, retransmissions, and connection overhead do not count as delivered bytes.

Code validation

All 36 portable tests passed on macOS and AWS Ubuntu 24.04 with AddressSanitizer and UndefinedBehaviorSanitizer. An independent UDP peer checks exact packet behavior, and actual endpoints verify integrity and loss recovery. Coverage includes initial window size, slow start, congestion avoidance, three duplicate ACKs, fast recovery, timeout reduction, flow-control limits, zero-window recovery, overlap/reordering, sequence wrap, handshake, peer isolation, and FIN teardown. Additional sanitizer checks cover changed packet/buffer parameters and 1 MiB bidirectional transfers. A deterministic drop verifies recovery before RTO. The experiment driver separately passed a 2 MiB byte-for-byte loopback transfer. Commands and validation results are in tests.txt.

Reproducibility

run_cp2_experiments.py records network settings, protocol constants, payload checksums, throughput, and raw congestion traces. build_cp2_report.py validates ten successful trials per loss rate, computes both regression forms, and renders this report. The data and calculations accompany the report in experiments/results. source.json and remote-builds.json verify matching C source and header hashes on both AWS nodes. The temporary instances are terminated after collecting the results; cleanup.json records removal of their volumes, security group, and keys.

References

Course Project 2 handout, checkpoint 2 requirements and Reno state diagram.
RFC 5681, TCP Congestion Control, sections 3.1-3.2: https://www.rfc-editor.org/rfc/rfc5681
RFC 9293, Transmission Control Protocol, section 3.8.6: https://www.rfc-editor.org/rfc/rfc9293